

20.5-4

Prove that for any directed graph G , the transpose of the component graph of G^T is the same as the component graph of G . That is, $((G^T)^{\text{SCC}})^T = G^{\text{SCC}}$.

20.5-5

Give an $O(V + E)$ -time algorithm to compute the component graph of a directed graph $G = (V, E)$. Make sure that there is at most one edge between two vertices in the component graph your algorithm produces.

20.5-6

Give an $O(V + E)$ -time algorithm that, given a directed graph $G = (V, E)$, constructs another graph $G' = (V, E')$ such that G and G' have the same strongly connected components, G' has the same component graph as G , and $|E'|$ is as small as possible.

20.5-7

A directed graph $G = (V, E)$ is *semiconnected* if, for all pairs of vertices $u, v \in V$, we have $u \rightsquigarrow v$ or $v \rightsquigarrow u$. Give an efficient algorithm to determine whether G is semiconnected. Prove that your algorithm is correct, and analyze its running time.

20.5-8

Let $G = (V, E)$ be a directed graph, and let $l: V \rightarrow \mathbb{R}$ be a function that assigns a real-valued label l to each vertex. For vertices $s, t \in V$, define

$$\Delta l(s, t) = \begin{cases} l(t) - l(s) & \text{if there is a path from } s \text{ to } t \text{ in } G, \\ -\infty & \text{otherwise.} \end{cases}$$

Give an $O(V + E)$ -time algorithm to find vertices s and t such that $\Delta l(s, t)$ is maximum over all pairs of vertices. (*Hint*: Use Exercise 20.5-5.)

Problems

20-1 *Classifying edges by breadth-first search*

A depth-first forest classifies the edges of a graph into tree, back, forward, and cross edges. A breadth-first tree can also be used to classify the edges reachable from the source of the search into the same four categories.

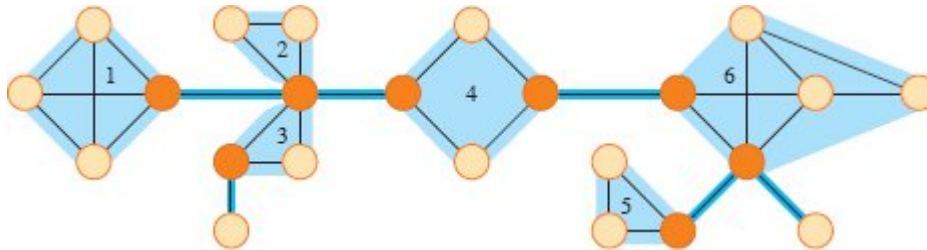


Figure 20.10 The articulation points, bridges, and biconnected components of a connected, undirected graph for use in Problem 20-2. The articulation points are the orange vertices, the bridges are the dark blue edges, and the biconnected components are the edges in the light blue regions, with a *bcc* numbering shown.

a. Prove that in a breadth-first search of an undirected graph, the following properties hold:

1. There are no back edges and no forward edges.
2. If (u, v) is a tree edge, then $v.d = u.d + 1$.
3. If (u, v) is a cross edge, then $v.d = u.d$ or $v.d = u.d + 1$.

b. Prove that in a breadth-first search of a directed graph, the following properties hold:

1. There are no forward edges.
2. If (u, v) is a tree edge, then $v.d = u.d + 1$.
3. If (u, v) is a cross edge, then $v.d \leq u.d + 1$.
4. If (u, v) is a back edge, then $0 \leq v.d \leq u.d$.

20-2 *Articulation points, bridges, and biconnected components*

Let $G = (V, E)$ be a connected, undirected graph. An **articulation point** of G is a vertex whose removal disconnects G . A **bridge** of G is an edge whose removal disconnects G . A **biconnected component** of G is a maximal set of edges such that any two edges in the set lie on a common simple cycle. Figure 20.10 illustrates these definitions. You can

determine articulation points, bridges, and biconnected components using depth-first search. Let $G_\pi = (V, E_\pi)$ be a depth-first tree of G .

a. Prove that the root of G_π is an articulation point of G if and only if it has at least two children in G_π .

b. Let v be a nonroot vertex of G_π . Prove that v is an articulation point of G if and only if v has a child s such that there is no back edge from s or any descendant of s to a proper ancestor of v .

c. Let

$$v.low = \min \begin{cases} v.d, \\ w.d : (u, w) \text{ is a back edge for some descendant } u \text{ of } v. \end{cases}$$

Show how to compute $v.low$ for all vertices $v \in V$ in $O(E)$ time.

d. Show how to compute all articulation points in $O(E)$ time.

e. Prove that an edge of G is a bridge if and only if it does not lie on any simple cycle of G .

f. Show how to compute all the bridges of G in $O(E)$ time.

g. Prove that the biconnected components of G partition the nonbridge edges of G .

h. Give an $O(E)$ -time algorithm to label each edge e of G with a positive integer $e.bcc$ such that $e.bcc = e'.bcc$ if and only if e and e' belong to the same biconnected component.

20-3 Euler tour

An **Euler tour** of a strongly connected, directed graph $G = (V, E)$ is a cycle that traverses each edge of G exactly once, although it may visit a vertex more than once.

a. Show that G has an Euler tour if and only if $\text{in-degree}(v) = \text{out-degree}(v)$ for each vertex $v \in V$.

- b. Describe an $O(E)$ -time algorithm to find an Euler tour of G if one exists. (*Hint*: Merge edge-disjoint cycles.)

20-4 Reachability

Let $G = (V, E)$ be a directed graph in which each vertex $u \in V$ is labeled with a unique integer $L(u)$ from the set $\{1, 2, \dots, |V|\}$. For each vertex $u \in V$, let $R(u) = \{v \in V : u \rightsquigarrow v\}$ be the set of vertices that are reachable from u . Define $\min(u)$ to be the vertex in $R(u)$ whose label is minimum, that is, $\min(u)$ is the vertex v such that $L(v) = \min \{L(w) : w \in R(u)\}$. Give an $O(V + E)$ -time algorithm that computes $\min(u)$ for all vertices $u \in V$.

20-5 Inserting and querying vertices in planar graphs

A **planar** graph is an undirected graph that can be drawn in the plane with no edges crossing. Euler proved that every planar graph has $|E| < 3|V|$.

Consider the following two operations on a planar graph G :

- $\text{INSERT}(G, v, \text{neighbors})$ inserts a new vertex v into G , where *neighbors* is an array (possibly empty) of vertices that have already been inserted into G and will become all the neighbors of v in G when v is inserted.
- $\text{NEWEST-NEIGHBOR}(G, v)$ returns the neighbor of vertex v that was most recently inserted into G , or NIL if v has no neighbors.

Design a data structure that supports these two operations such that NEWEST-NEIGHBOR takes $O(1)$ worst-case time and INSERT takes $O(1)$ amortized time. Note that the length of the array *neighbors* given to INSERT may vary. (*Hint*: Use a potential function for the amortized analysis.)

Chapter notes

Even [137] and Tarjan [429] are excellent references for graph algorithms.

Breadth-first search was discovered by Moore [334] in the context of finding paths through mazes. Lee [280] independently discovered the same algorithm in the context of routing wires on circuit boards.

Hopcroft and Tarjan [226] advocated the use of the adjacency-list representation over the adjacency-matrix representation for sparse graphs and were the first to recognize the algorithmic importance of depth-first search. Depth-first search has been widely used since the late 1950s, especially in artificial intelligence programs.

Tarjan [426] gave a linear-time algorithm for finding strongly connected components. The algorithm for strongly connected components in Section 20.5 is adapted from Aho, Hopcroft, and Ullman [6], who credit it to S. R. Kosaraju (unpublished) and Sharir [408]. Dijkstra [117, Chapter 25] also developed an algorithm for strongly connected components that is based on contracting cycles. Subsequently, Gabow [163] rediscovered this algorithm. Knuth [259] was the first to give a linear-time algorithm for topological sorting.

¹ We distinguish between gray and black vertices to help us understand how breadth-first search operates. In fact, as Exercise 20.2-3 shows, we get the same result even if we do not distinguish between gray and black vertices.

² Chapters 22 and 23 generalize shortest paths to weighted graphs, in which every edge has a real-valued weight and the weight of a path is the sum of the weights of its constituent edges. The graphs considered in the present chapter are unweighted or, equivalently, all edges have unit weight.

³ It may seem arbitrary that breadth-first search is limited to only one source whereas depth-first search may search from multiple sources. Although conceptually, breadth-first search could proceed from multiple sources and depth-first search could be limited to one source, our approach reflects how the results of these searches are typically used. Breadth-first search usually serves to find shortest-path distances and the associated predecessor subgraph from a given source. Depth-first search is often a subroutine in another algorithm, as we'll see later in this chapter.